

2.2 Generation Prompts

For the *Machine-Written Machine-Humanized* class, examples of prompts include *Rewrite this text to make it sound more natural and human-written* or *“Rephrase this text to be easy to understand and personable.”* For the *Human-Written Machine-Polished* class, we used prompts such as *“Paraphrase the provided text.”* or *“Rewrite this text so that it is grammatically correct and flows nicely.”* Additionally, we introduced a trailing prompt appended to each randomly selected prompt to prevent undesirable text that the LLM may prepend to its output, e.g., *“Only output the text in double quotes with no text before or after it. Text: {} Your response:”*. We used 5-6 prompts per domain to generate data for the *Machine-Written Machine-Humanized* and *Human-Written Machine-Polished* classes. In addition to the *Machine-Generated* class, we used the original prompts from the M4GT-Bench dataset.

2.3 API Tools & Data Cleaning

For data generation, we used multiple APIs from OpenAI, Gemini, Groq, and DeepInfra, to generate a total of 303,110 texts for the three LLM-dependent classes. For each of the two new class generations, we limited the text length to 1,500 words in order to accommodate the context length restrictions of some smaller LLMs and to efficiently manage time and costs.

The output of the LLMs occasionally included formatting such as “Here is the paraphrased text:” and “Sure!” despite instructions in the trailing prompt to exclude any additional output. We removed these phrases in the post-processing with two considerations. On the one hand, this naturally occurs in real-world applications, i.e., humans will remove these irrelevant phrases when they use the target content. Moreover, the presence of these indicative artifacts could impact the detectors’ generalization and the quality of the dataset, given that they are potentially unique for a specific text class.

3 Detection Models

We trained three detectors by fine-tuning RoBERTa (Liu et al., 2019), DeBERTa (He et al., 2021), and DistilBERT (Sanh et al., 2019). DeBERTa is built upon BERT and RoBERTa by incorporating disentangled attention mechanisms and an enhanced mask decoder, which improves word representation.

Dataset	Detector	Learning rate	Weight Decay	Epochs	Batch Size
arXiv	RoBERTa	2e-5	0.01	10	16
	DistilBERT	2e-5	0.01	10	16
OUTFOX	RoBERTa	2e-5	0.01	10	16
	DistilBERT	2e-5	0.01	10	16
Full Dataset	RoBERTa	5e-5	0.01	10	32
	DeBERTa	5e-5	0.01	10	32

Table 2: **Hyper-parameter values** across the models.

Eventually, in the demo, we used DistilBERT, which is a compact and fast variant of BERT: 60% faster and 40% smaller, while retaining 97% of BERT’s language understanding capabilities.

Table 2 shows the values of the hyper-parameters for each model. We used RoBERTa and DistilBERT in our domain-specific experiments. However, due to the inferior performance of DistilBERT to RoBERTa in our preliminary trials, we substituted DistilBERT with DeBERTa in the following experiments (DeBERTa is superior to RoBERTa).

4 Experiments and Evaluation

The previous studies have shown that the accuracy of detectors drops substantially when testing on out-of-domain examples (Wang et al., 2024b). To alleviate this, we propose three strategies: (i) train multiple domain-specific detectors, each specifically responsible for detecting inputs from one domain, (ii) train one universal detector using more training data across various domains, and (iii) leverage domain-adversarial neural network (DANN) for domain adaption.

4.1 Domain-Specific Detectors

We fine-tuned RoBERTa and DistilBERT using the data from arXiv and OUTFOX, using a ratio of training, validation, and test sets of 70%:15%:15%. The results are shown in Table 3. We can see that both RoBERTa and DistilBERT performed well on OUTFOX. Overall, RoBERTa is more robust over diverse domains, with accuracy greater than 95% on both domains, with a small number of mis-classifications occurring between classes with overlapping features, such as Machine-Generated vs. Human-Written, vs. Machine-Polished classes, as the confusion matrices in Figure 2 show.

However, in this setup, the users need to first specify the domain of the input text, which is an extra effort. To mitigate this, we further trained a universal model that does not require the user to select the domain.

Detector	Test Domain	Prec	Recall	F1-macro	Acc
RoBERTa	arXiv	95.82	95.79	95.79	95.79
	OUTFOX	95.67	95.43	95.53	95.65
DistilBERT	arXiv	88.98	87.97	87.93	87.79
	OUTFOX	96.66	96.65	96.65	96.65

Table 3: **Domain-specific performance** for RoBERTa and DistilBERT on arXiv and OUTFOX.

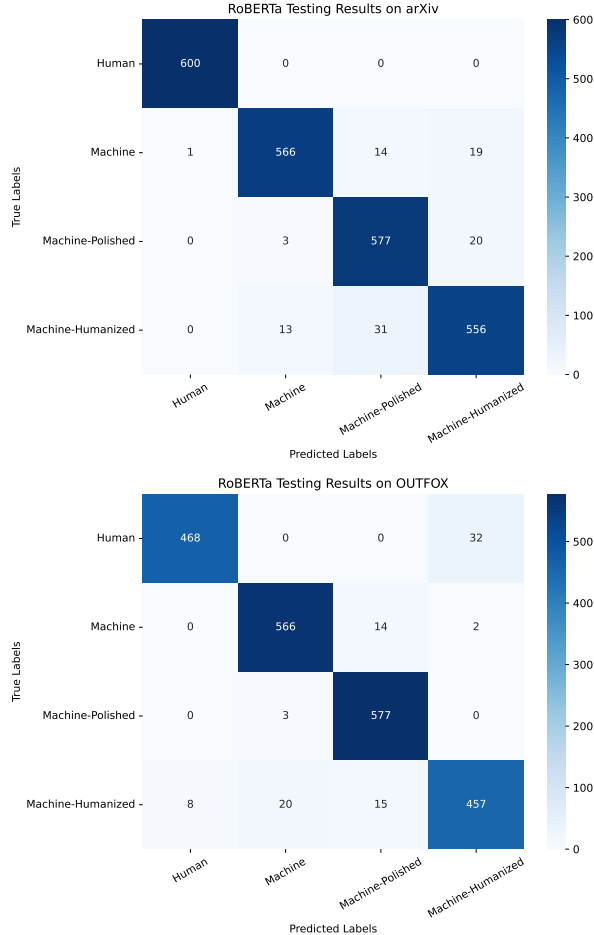


Figure 2: Domain-specific confusion matrix for RoBERTa on arXiv (top) and on OUTFOX (bottom).

4.2 Universal Detectors

We fine-tuned RoBERTa and DeBERTa using the full dataset; the data distribution for this is shown in Table 4. To reduce data imbalance and prevent the detector from favoring any particular class, we excluded some of the original data. The evaluation results in Table 5 indicate that DeBERTa consistently outperforms RoBERTa across all evaluation measures we use. Therefore, we deployed the fine-tuned DeBERTa as the backend detection model for our demo.

Domain	Human	Machine-Generated	Machine-Polished	Machine-Humanized
arXiv	15,998	18,000	18,000	18,000
Reddit	16,000	18,904	18,904	18,904
wikiHow	15,999	22,601	22,601	22,601
Wikipedia	14,333	22,615	22,615	22,615
PeerRead	2,847	4,684	4,684	4,684
Outfox	14,043	17,000	17,000	17,000

Table 4: **Distribution** of the data used for fine-tuning **universal detectors** based on RoBERTa and DeBERTa.

Detector	Prec	Recall	F1-Macro	Acc
RoBERTa	94.79	94.63	94.65	94.62
DeBERTa	95.71	95.78	95.72	95.71

Table 5: **Detector performance** on the full dataset.

4.3 DANN-Based Detector

In our domain-specific experiments above, we achieved strong performance when the domain of the text was provided. However, in cross-domain evaluation, the performance is sub-optimal as previous work has suggested (Wang et al., 2024b,c). In real-world scenarios, the domain would not always be specified, and thus we need a classifier that is as domain-independent as possible. Thus, we investigated the use of *domain adversarial neural networks* (Ganin et al., 2017) to train a domain-robust detector.

DANN was initially designed to achieve domain adaptation by aligning representations across different domains with three major components:

- **Representation Extractor:** which builds a representation of the input data; here, we use RoBERTa.
- **Label Predictor:** to predict the class labels based on the representation; it is trained using labeled data from the source domain.
- **Domain Classifier:** connected to the representation via a *gradient reversal layer (GRL)*, it distinguishes between the source and the target domains. It multiplies the gradient by a negative constant during back-propagation, promoting domain-invariant representation.

The network is trained using standard back-propagation and stochastic gradient descent, optimizing the label classification loss while intentionally confusing the model regarding the domain by reversing the gradient from the domain classifier. This reduces the label classification loss while increasing the domain classification one.

Detector	Prec	Recall	F1-macro	Acc
RoBERTa	94.79	94.63	94.65	94.62
DANN+RoBERTa	96.30	95.54	96.06	95.24

Table 6: Comparing domain-specific RoBERTa vs. DANN+RoBERTa. The latter outperforms the former across all measures, indicating that decoupling the model from domain-specific representation is beneficial.

As a result, the Domain-Adversarial Neural Network (DANN) yields a representation that is independent of the domain. In our experiments, we trained the DANN to predict our four classes and to be as confused as possible when predicting the six sources/domains. The results are shown in Table 6. We can see that using domain adversarial training on top of RoBERTa-enhances the overall performance compared to just fine-tuning RoBERTa as in Section 4.2. This suggests that decoupling the model from domain-specific representation leads to an improvement in its overall performance.

4.4 Comparison to Existing Systems

There are several previously proposed systems for detecting machine-generated text, such as GPTZero,² ZeroGPT,³ and Sapling AI detector,⁴ but none of them supports four classes. GPTZero is the only one that goes beyond binary classification: it adds a *mixed text*; however, it limits users to only 40 free runs per day or 10,000 words per month for registered accounts. Thus, we could not perform comparison on our entire test dataset. Instead, we randomly sampled 60 machine-generated texts and 60 human texts (10 per source) per source. In this binary classification setting, LLM-DetectAIve achieved 97.50% accuracy, outperforming GPTZero, ZeroGPT, and Sapling AI, with 87.50%, 69.17%, and 88.33%, respectively.

4.5 Generalization Evaluation

To evaluate the generalization ability of our detector on unseen domains and generators, we experimented with testing on two additional datasets: MixSet (Zhang et al., 2024) and IELTS essays written by individuals for whom English is a second language.⁵

²<https://gptzero.me/>

³<https://www.zerogpt.com/>

⁴<https://sapling.ai/ai-content-detector>

⁵https://huggingface.co/datasets/chillies/IELTS_essay_human_feedback

Dataset	Prec	Recall	F1-macro	Acc
IELTS	63.74	66.91	66.55	66.91
MixSet	59.18	64.25	54.95	60.08

Table 7: **Cross-domain evaluation** of our detector on unseen domains and generators: IELTS and MixSet.

For the IELTS essays, after deduplication, we randomly sampled 300 (essay problem statement, human-written essay) pairs, and then we produced the corresponding machine-written essays using the problem statements based on Llama3.1-70B. We further generated Machine-Written Machine-Humanized and Human-Written Machine-Polished. For MixSet, the original dataset contains a total of 3,600 examples, with 300, 300, 600, and 2,400 examples for Human-Written, Machine-Generated, Machine-Written Machine-Humanized and Human-Written Machine-Polished, respectively. It involves models such as Llama2-70B and GPT-4, and text covering domains of email content, news, game reviews, and so on.

The results are shown in Table 7, where we can see that the detector performs much worse on unseen domains and generators, compared to in-domain and in-generator cases. The performance on IELTS is better than on MixSet. This can be attributed to the inclusion of the OUTFOX data (English native-speaker student essays) in the training data, while the domains and the generators in MixSet are not in the training set. The low generalization performance suggests challenges in adapting black-box detectors to the diverse domains and generators in real-world applications.

5 Demo Web Application

Our demo web application has two interfaces: (i) an interface for fine-grained MGT detection, and (ii) a playground for users.

5.1 Automatic Detection

The automatic detection interface is shown in Figure 1 (top). It allows users to input a text, and then the system responds with the class that the text belongs to. To ensure the prediction accuracy, the length of the submitted text is constrained to 50-500 words since the performance of our detectors drops significantly for shorter texts. Longer texts will be truncated, as we are limited by the context window size of the BERT-like transformers we use.